

Systemes Temps Réel

INTRODUCTION/LINUX



Plan

❑ Partie 1

- Introduction au temps réel et outils associés
- Linux & Temps Réel
- I/O Systems (gestion des entrées/sorties)/communication de processus

❑ Partie 2

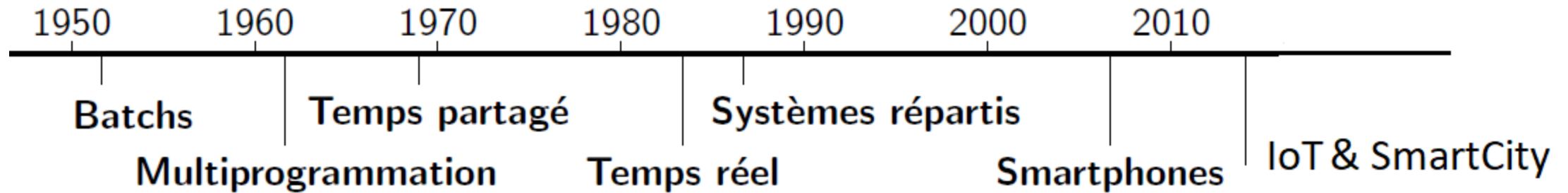
- ordonnancement TR

❑ Partie 3

- Temps-réel strict - Extensions RTAI et API
- Temps-réel strict Xenomai et API
- Reconfiguration de Xenomai
- Multitâches temps réel



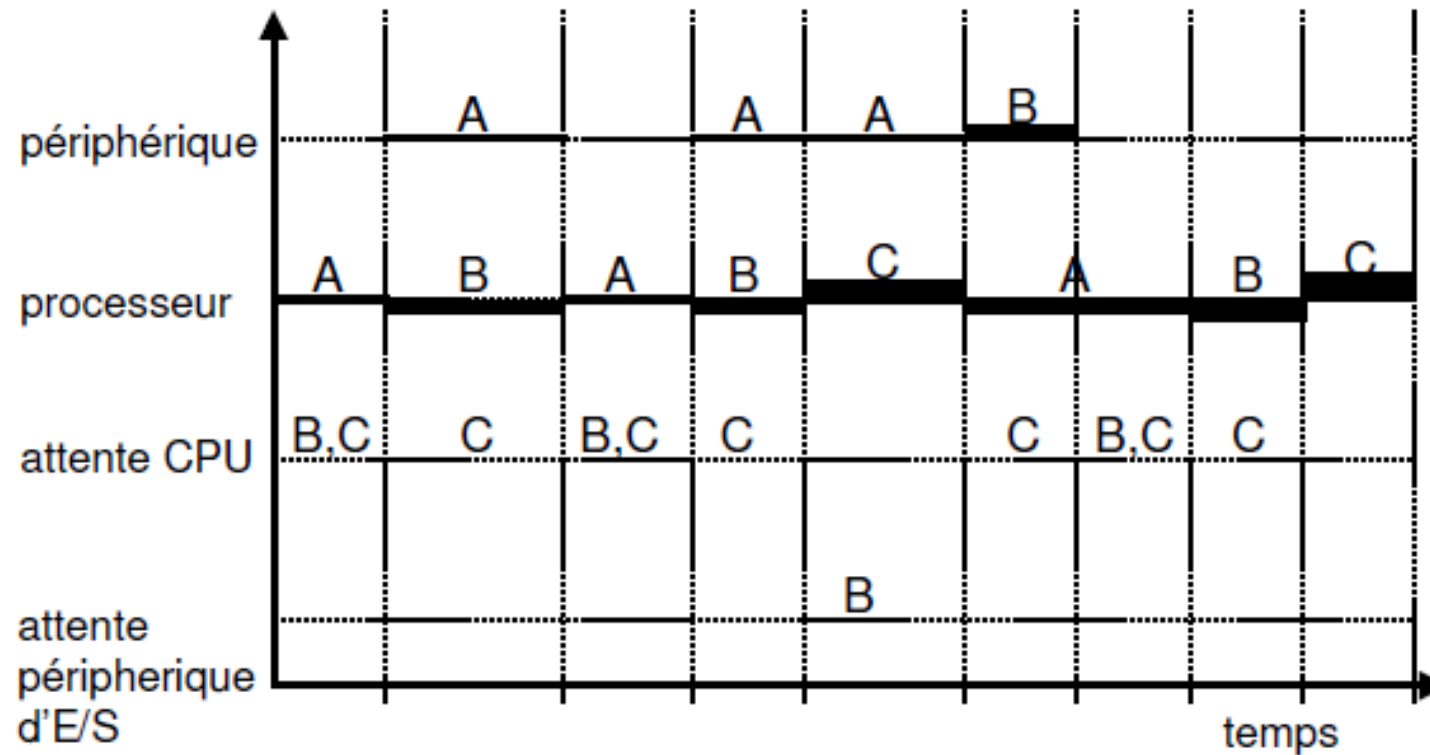
Un peu d'histoire



Évolution du SE

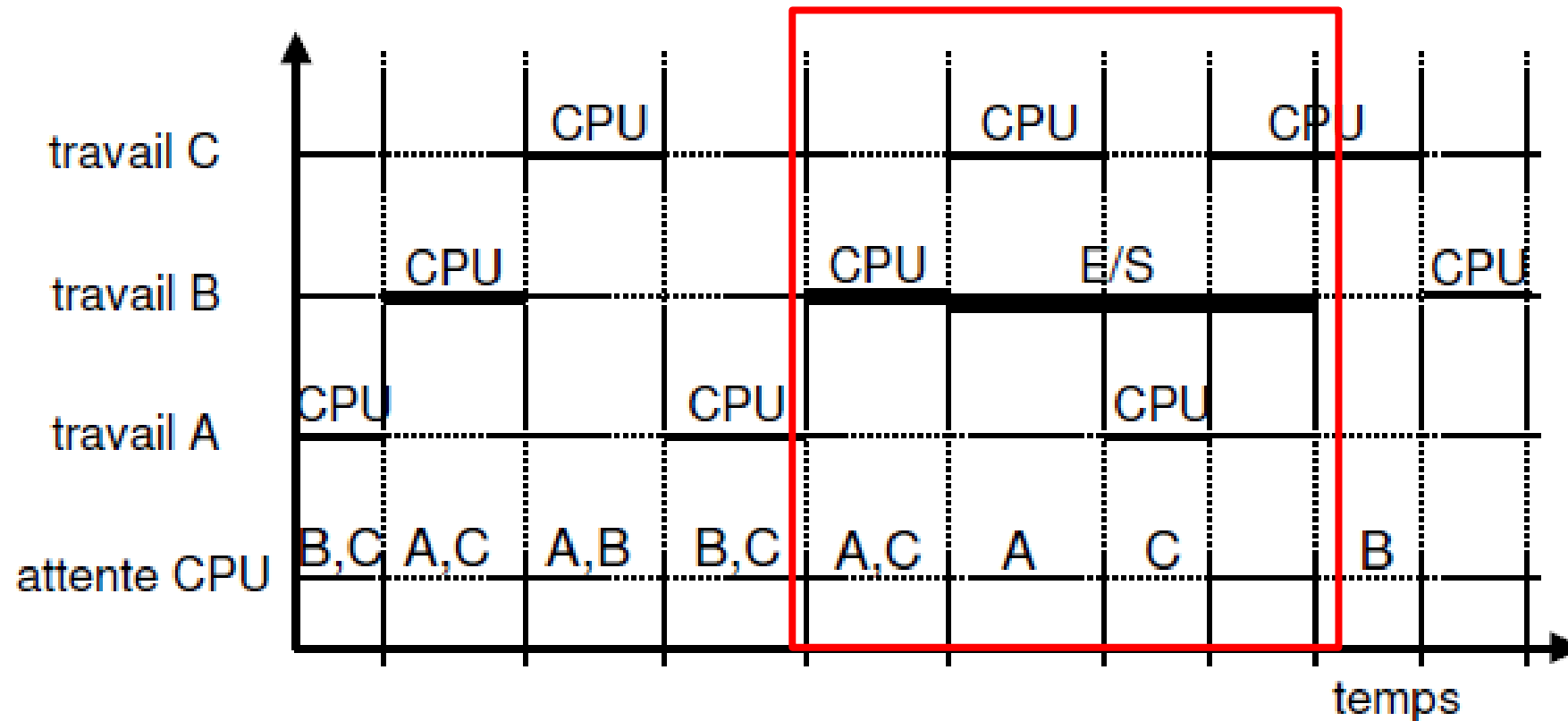
multiprogrammation & traitement par lots

Exemple : A,B et C 3 travaux qui démarrent leur exécution simultanément



Commutation des processus

Multiprogrammation & Temps partagé



Temps partagé/exécution parallèle(quantum)



Systèmes temps réel (STR)

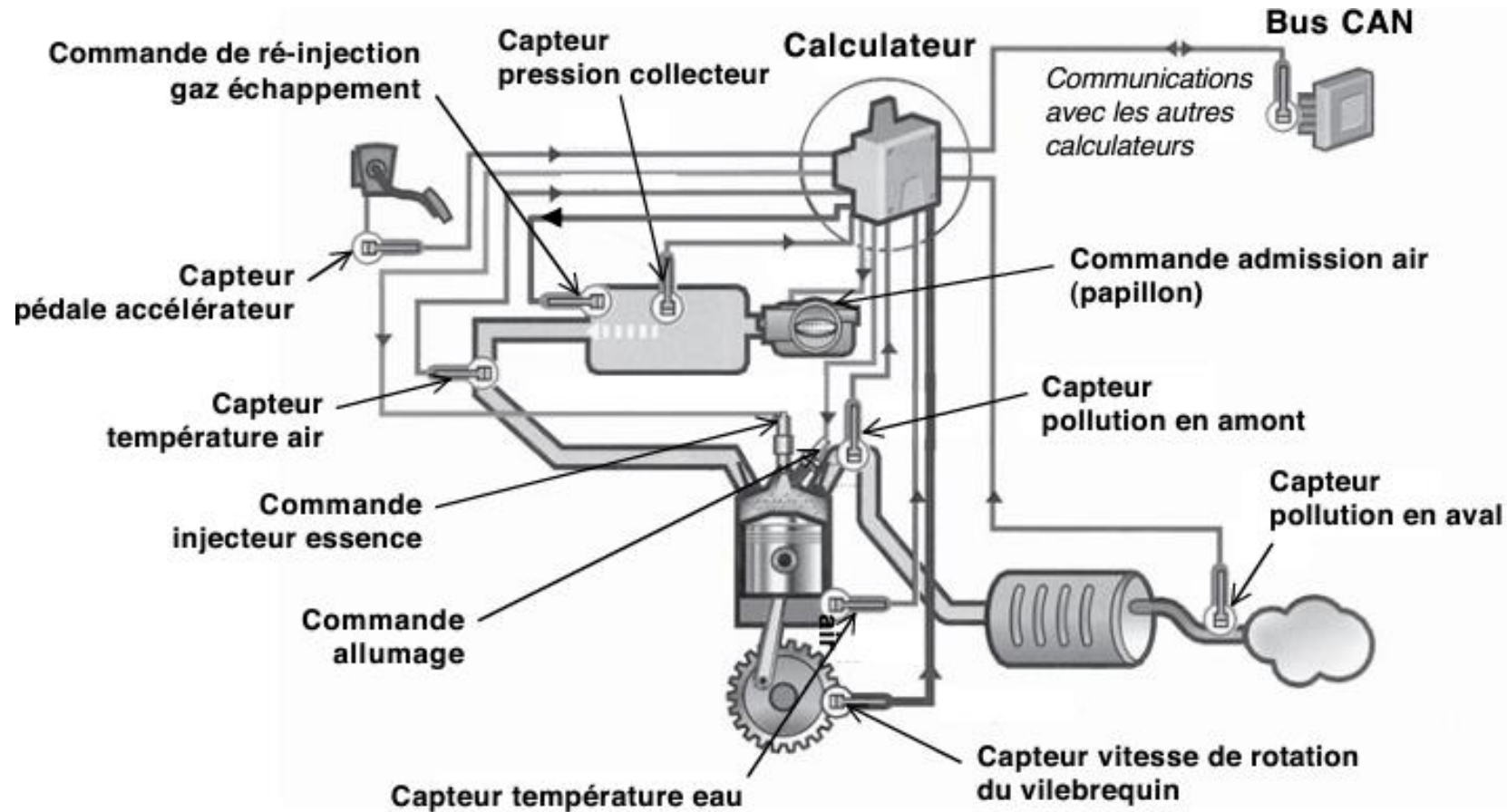
- ❑ Un système temps réel est un système informatique dont le fonctionnement est assujéti à l'évolution dynamique d'un procédé extérieur qui lui est connecté et dont il doit contrôler le comportement.
- ❑ La correction d'un STR dépend non seulement de la justesse des calculs mais aussi du temps auquel les résultats sont produits [Stankovic 1988] (contraintes de temps).
- ❑ Un STR n'est pas un système « qui va vite / rapide » mais un système qui satisfait des contraintes temporelles (les contraintes de temps).



Caractéristiques du temps réel

- ❑ Taille et complexité
 - Un système temps réel interagit avec un environnement extérieur souvent complexe et en évolution
 - Il doit respecter des échéances temporelles, garantir une fiabilité permanente
 - Il doit pouvoir interagir avec différents types d'éléments matériels.
- ❑ Implémentation efficace: restrictions dans l'utilisation des constructions du langage
- ❑ Certification: garantir un fonctionnement conforme aux spécifications
- ❑ Simulation, prototypage: vérification a priori de la conception
- ❑ Plate-formes d'essai: vérification a posteriori du bon fonctionnement

Exemple d'application



contrôle-commande d'un moteur à combustion.



STR / Temps réel Souple

❑ Temps Réel Souple (*Soft Real Time*)

Retard de traitement ou perte de résultat sans conséquence catastrophique.

❑ Exemple:

Systèmes multimédia, Guichet Automatique,...

→ Systèmes d'exploitation classiques à temps partagé.



Systèmes temps réel/ Temps réel Dur (TRD)

❑ TDR :

événements traités trop tardivement ou perdus provoquent des conséquences catastrophiques.

❑ Les TRD ne tolèrent qu'une gestion stricte et bornée du temps.

❑ Exemples:

- les contrôles de processus industriels sensibles comme la régulation des centrales nucléaires
- les systèmes embarqués utilisés dans l'aéronautique.
- Les systèmes de sécurité automobile



Systèmes temps réel/ Temps réel Dur (TRD)

❖ Caractéristiques

- systèmes garantissent un temps maximum d'exécution pour chaque tâche.
- Répartition totalitaire du temps CPU entre tâches.
- Un système temps réel doit être prévisible.
- Les contraintes temporelles pouvant allant jusqu'à quelques μs .



Systèmes temps réel/Outils de développement

- ❑ Langages de bas niveau: C/C++, assembleur, Ada,...
 - Largement diffusés et utilisés à ce jour
 - Accès direct aux ressources de bas niveau
 - Idéal pour les I/O (entrées/sorties)
 - Doit être couplé avec les services du système (synchronisation, ordonnancement)
 - Recours aux librairies



Exemples de SETR (Wikipédia)

Exemples de systèmes d'exploitation temps réel [\[modifier \]](#) [\[modifier le code \]](#)

- Adeos
- ART Linux
- ChorusOS
- ContikiOS
- eCos
- ELinOS
- ERIKA Enterprise
- FreeRTOS
- HipperOS¹
- iRmx
- ITRON
- linux-rt²,
- LynxOS
- MPE [\(en\)](#)
- MicroC/OS-II (ou μ C/OS-II)
- nOS³
- Nucleus
- OS-9
- OSE
- OSEK/VDX
- pSOS
- PikeOS
- QNX
- RedHawk
- RSX-11
- RT-11
- RT-LAB
- HP-1000/RTE [\(en\)](#) de HP⁴
- RTAI
- RTEMS
- RTLinux ou *Real Time Linux*
- RTOS-UH
- RTX ou *Real Time eXtension to Windows*
- SCIOPTA
- Symbian OS
- VRTX
- VxWorks
- Windows CE
- Xenomai
- XNU :
 - Darwin
 - OS X
 - iOS
- Yasos

Domaines/Systèmes temps réel



Systèmes Temps Réel



Les STR Et Linux

- ❑ L'apparition des OS libres dans l'industrie et dans l'embarqué modifie la donne !
- ❑ Linux est utilisable dans l'industrie

Mais

- Linux n'est pas TR
- Linux peut être modifié pour être TR (PREEMPT-RT, Xenomai)
- Il existe des systèmes TR légers et libres (RTEMS, FreeRTOS,...)



Les STR Et Linux

- ❑ Linux n'est pas un SETR dur car :
 - Le noyau possède de longues sections de code (non interruptible).
 - Le noyau n'est pas **préemptible** (structure monolithique).
 - Le noyau Linux n'est pas **préemptible** durant le service ISR.
 - Temps de latence important et non borné!!



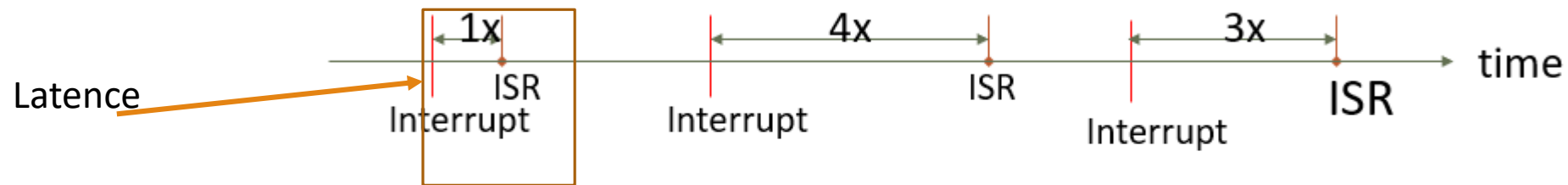
Les STR Et Linux?

- ❑ Linux standard n'est pas un SETR dur car :
 - Ordonnancement à approche égalitaire.
 - Le noyau Linux est SETR mou (réactivité >100 ms).
 - Solutions:
 - ✓ Linux Temps Réel mou (préemptifs) par patches (ADEOS ~ 100µs)
 - ✓ Linux Temps Réel dur pour une réactivité de ~10µs (xenomai, RTAI, ...)

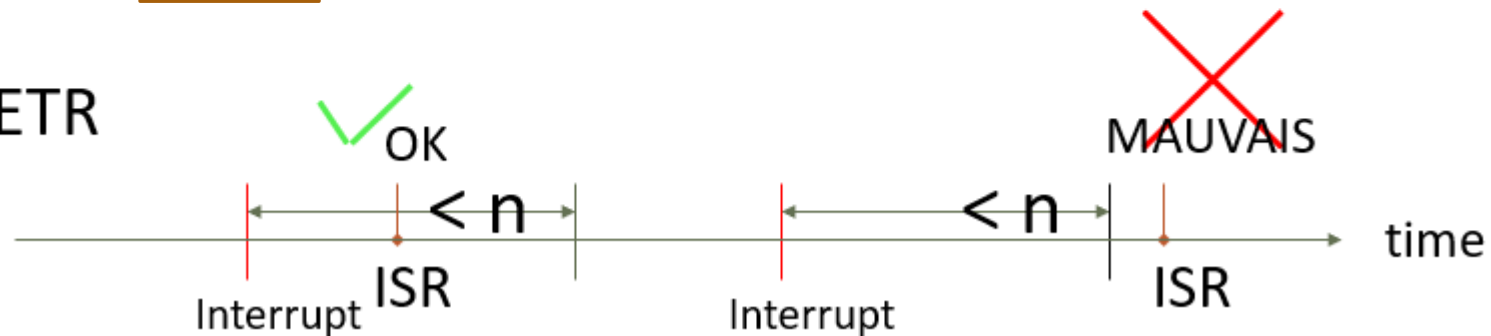
Les Systèmes Temps Réel Et Linux?

Comparaison

- Linux



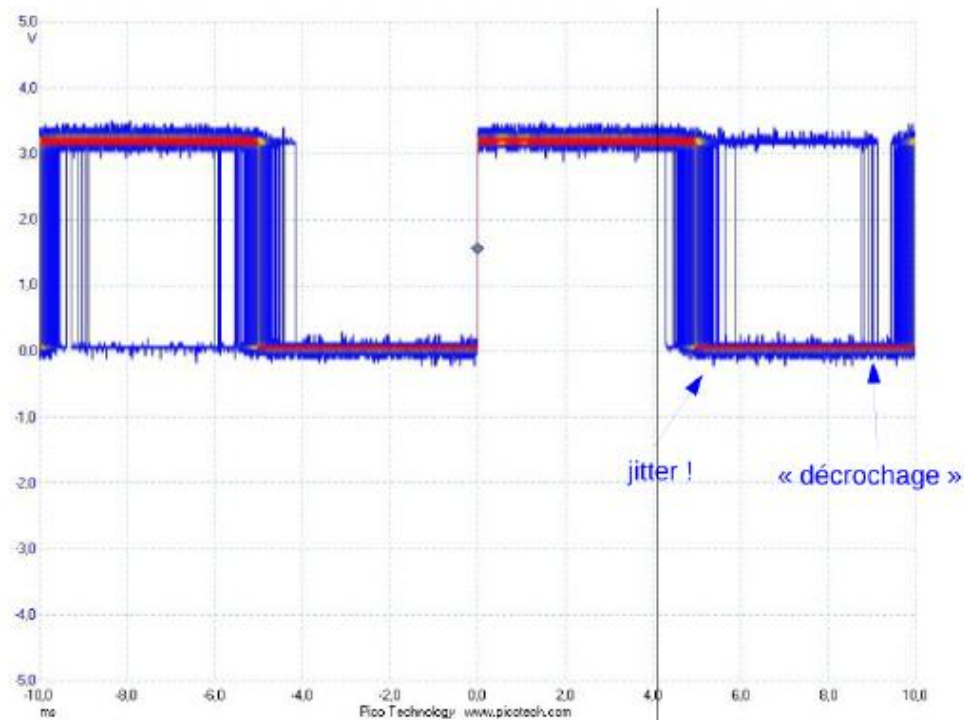
- SETR



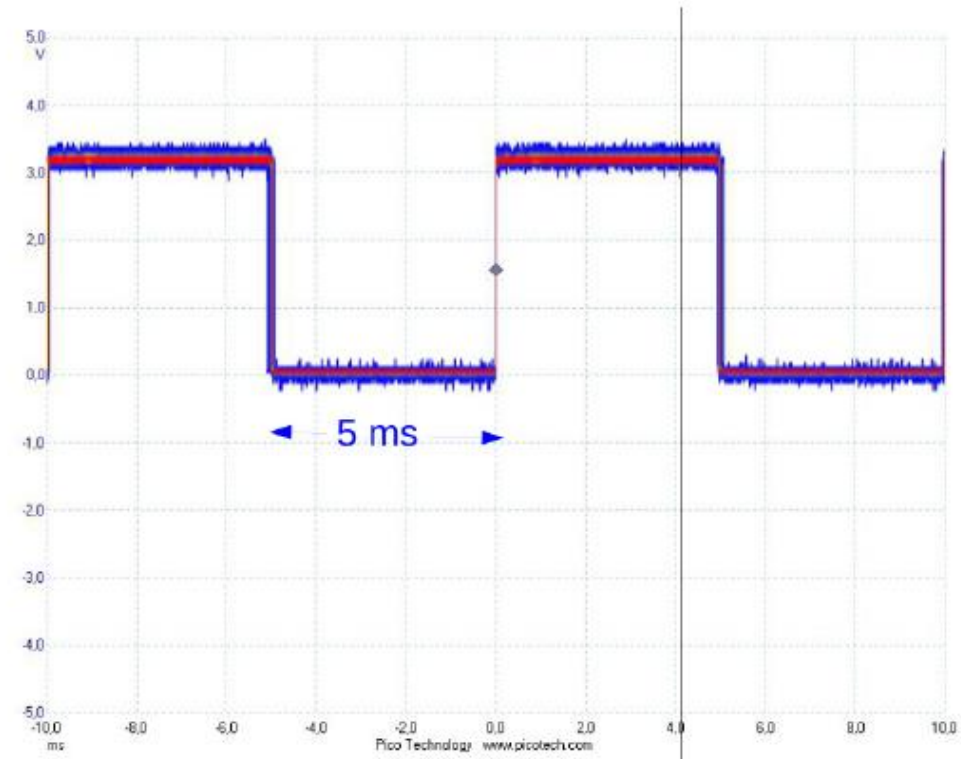
n : délai maximum

Les Systèmes Temps Réel Et Linux?

Sur un système chargé :



Noyau standard



Avec co-noyau Xenomai

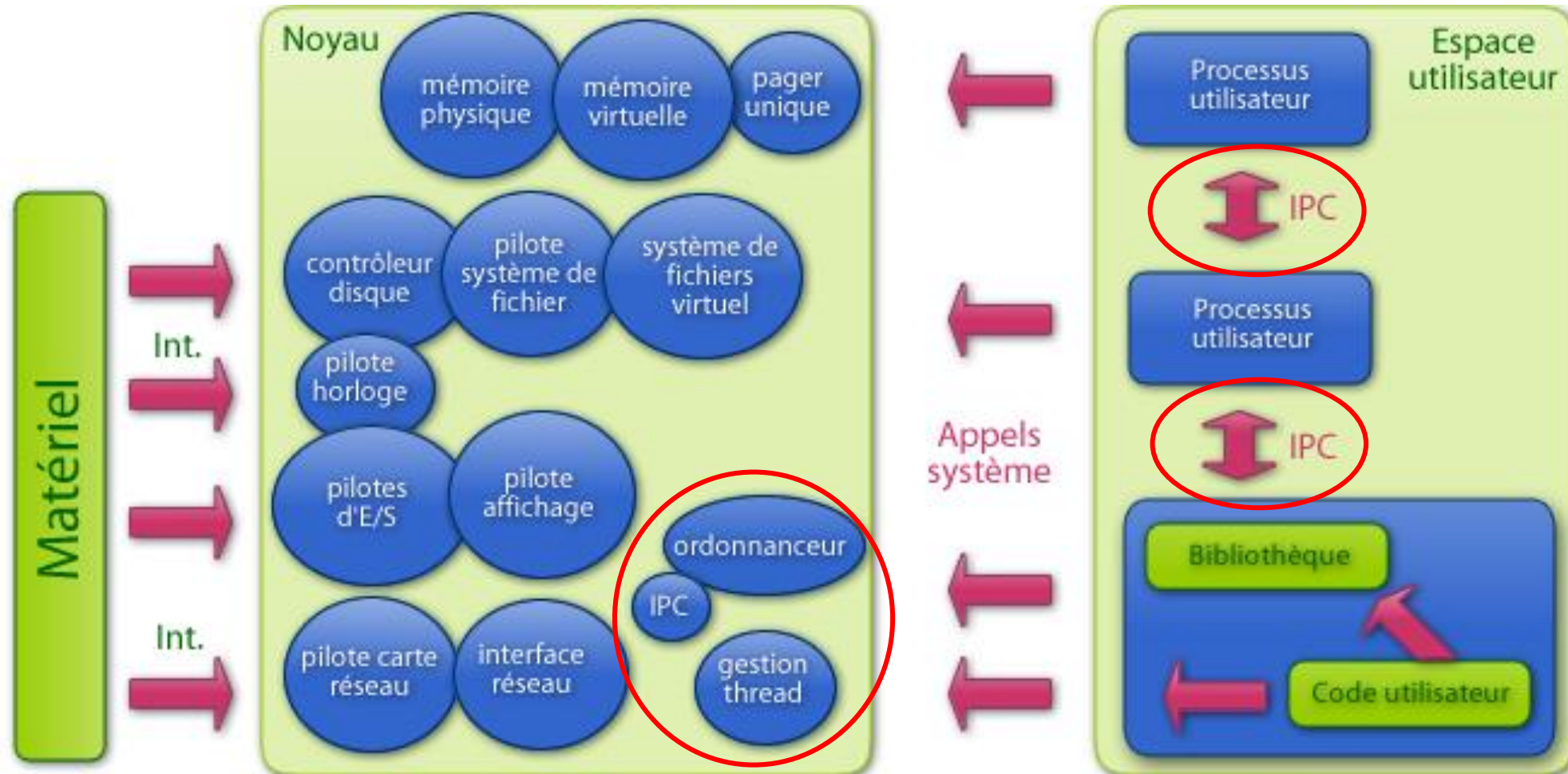


Les Systèmes Temps Réel Et Linux?

???

Linux standard →→ SETR

SE Linux/rappel

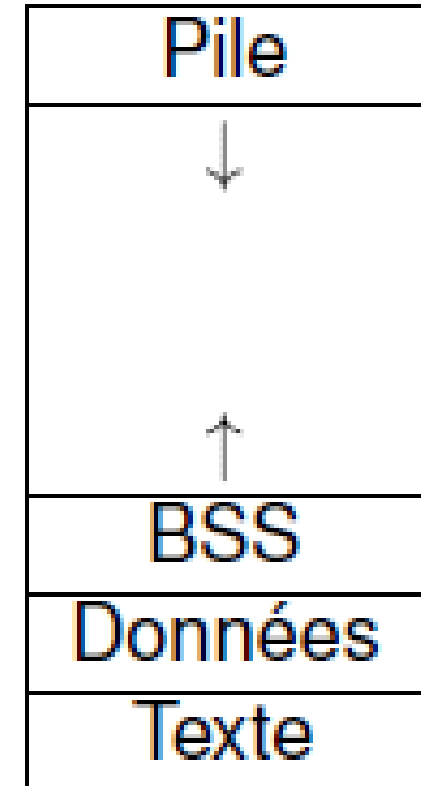


SE Linux/Processus

□ **processus** est un programme :

- Le segment de texte:
- Le segment de données/BSS (Block Started by Symbol):
- Le segment de pile:

Chaque processus possède un numéro qui lui est propre, le **pid**



SE Linux/Processus

□ pid du processus

- numéro unique (pid).

$$0 \leq \text{pid} \leq 32767$$

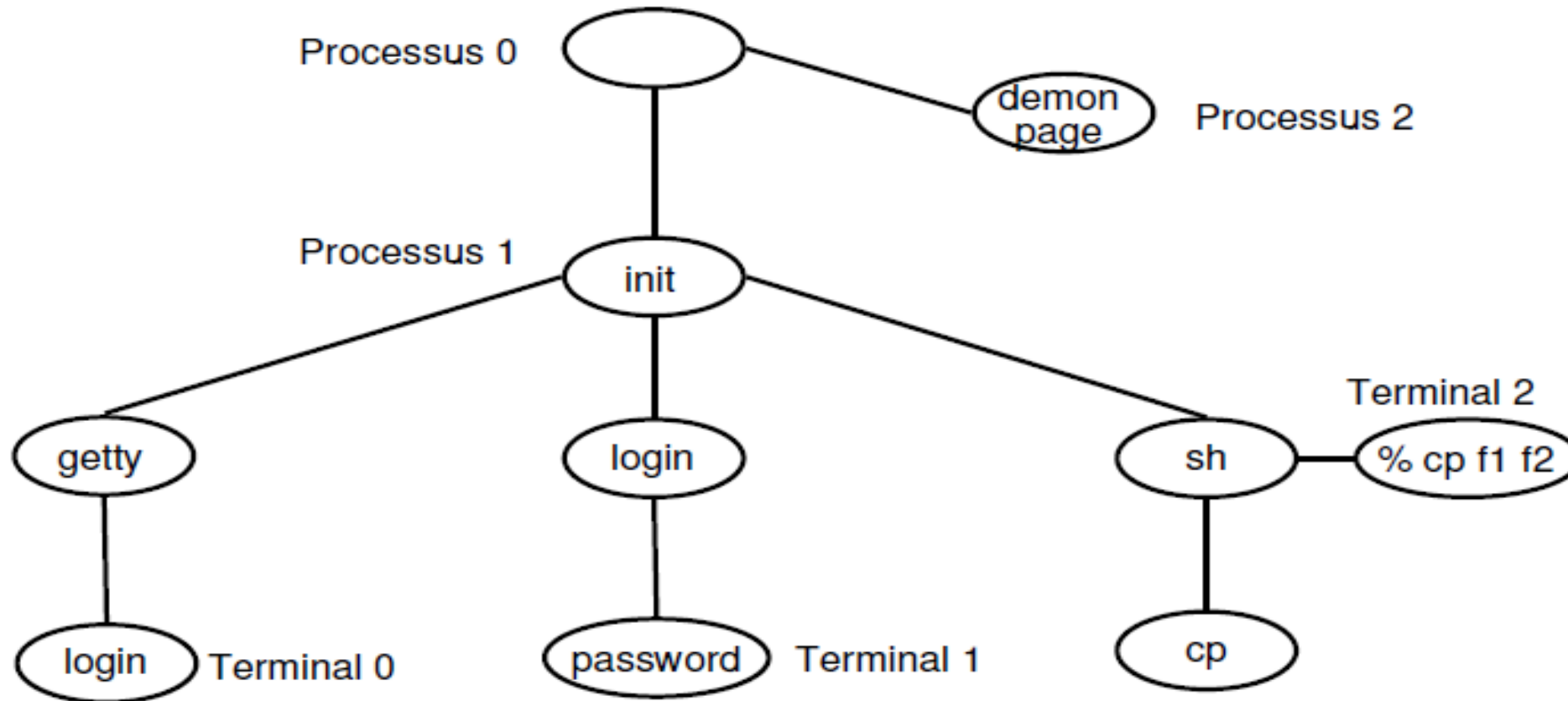
- Si $\text{pid} > 32767$, le noyau doit alors recycler des pid.

- Récupérer le pid par appel système :

int getpid()

- cmd ps: permet d'afficher les processus en cours d'exécution (ps tree, ps -u)

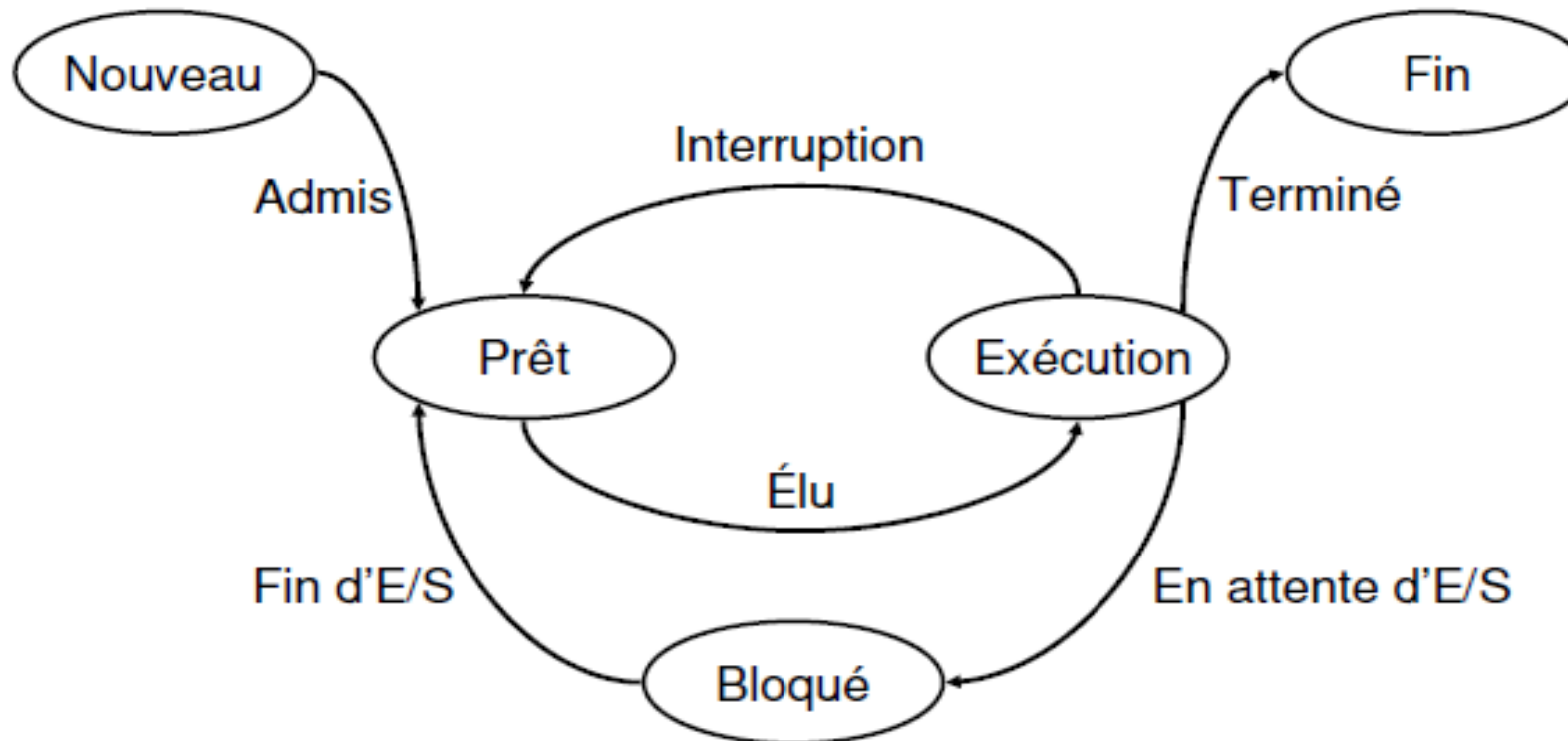
SE Linux/Processus (arbre de processus)



Démarrage de linux (& pid des processus)

SE Linux/Processus/États

- Le modèle à trois états est complété par deux états supplémentaires : **Nouveau** et **Fin**



SE Linux/Processus/Implémentation de processus

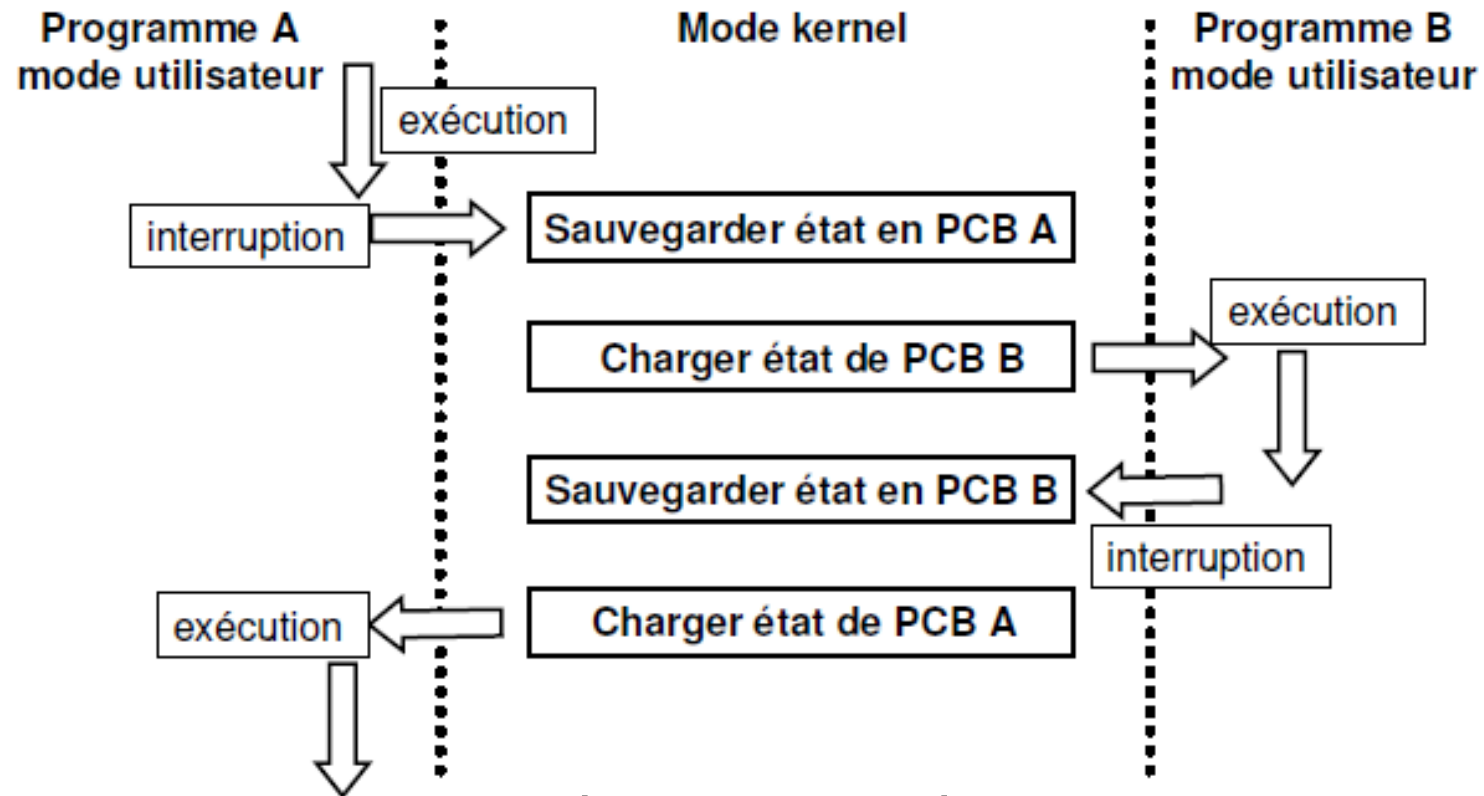
- Le système d'exploitation sauvegarde plusieurs informations dans des structures de données.

Gestion des processus	Gestion de la mémoire	Gestion des fichiers
Registres Compteur ordinal (co) Priorité Pointeur de pile (sp) État du processus Date de lancement Temps UCT utilisé Temps UCT des fils Signaux pid ppid	Pointeur seg. code Pointeur seg. données Pointeur seg. pile	Répertoire racine Répertoire de travail fd uid gid

PCB: Process Control Block

SE Linux/Processus/Implémentation de processus

Exemple



Le changement de contexte.

SE Linux/Processus/Appels système/gestion de processus

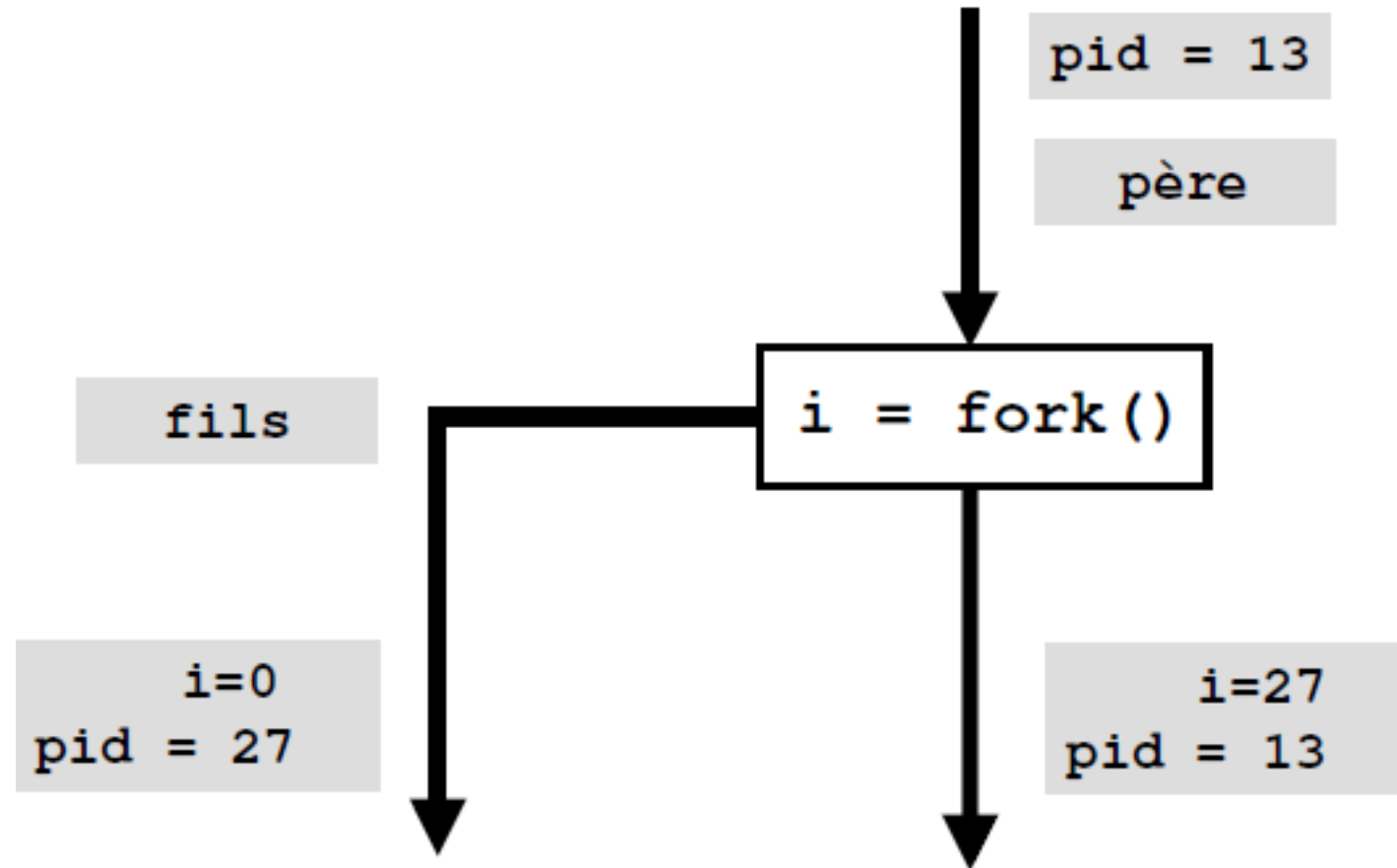
□ Gestion des processus

- **pid_t= fork()** : Création de processus fils.
- **pid_t= wait()** : Attendre la terminaison d'un processus.
- **void exit()/kill()** : Finir l'exécution d'un processus.
- **pid_t =getpid()** : Retourne l'identifiant du processus.
- **pid_t= getppid()** : Retourne l'identifiant du processus père.

le type pid_t correspond normalement à un long int.

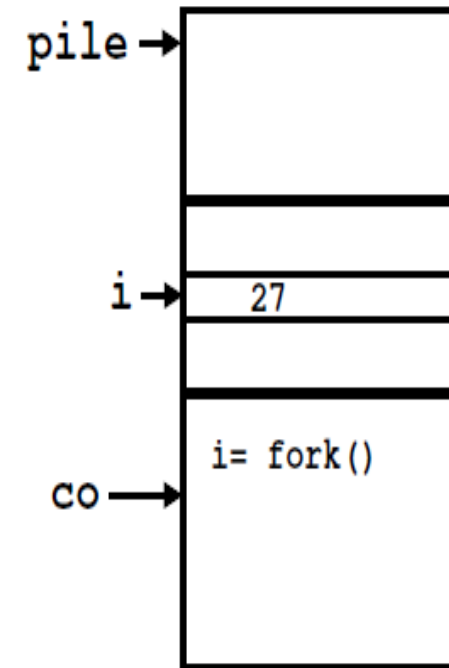
SE Linux/Processus/création (fork)

Exemple 1

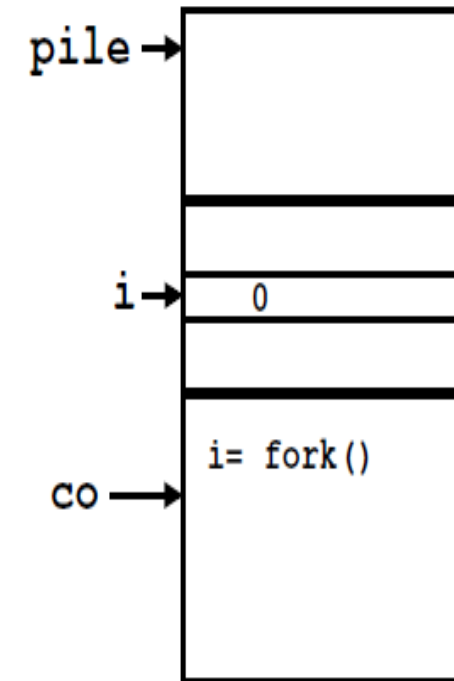


SE Linux/Processus/Création (fork)

- `fork()` crée une copie exacte du processus original,
- Les deux processus père et fils exécutent le même code.
- Pour les différencier, la valeur de retour de `fork()`, qui peut être (i) :
 - ✓ 0 pour le processus fils
 - ✓ >0 pour le processus père et qui correspond au pid du fils
 - ✓ <0 si la création de processus a échoué.



Processus père



Processus fils

SE Linux/Processus/Création du fis (fork)

Exemple 2

Le Papa

Descripteurs de fichiers

1: (Stdout) Position = 0

Variables globales

const char* quisuisje = NULL;

```

int main()
{
→ pid_t pid;
  quisuisje = "Le pere";
  pid = fork();
  if(pid == 0){
    quisuisje = "Le fils";
    printf("Je suis %s", quisuisje);
  }
  else{
    printf("Je suis %s", quisuisje);
    wait(NULL);
  }
  return 0;
}

```

Duplication
du code



Le Papa

Descripteurs de fichiers

1: (Stdout) Position = 0

Variables globales

const char* quisuisje = "Le pere";

```

int main()
{
  pid_t pid;
  quisuisje = "Le pere";
  pid = fork(); //pid = 1000
  if(pid == 0){
    quisuisje = "Le fils";
    printf("Je suis %s", quisuisje);
  }
  else{
→ printf("Je suis %s", quisuisje);
    wait(NULL);
  }
  return 0;
}

```

Le Fiston

Descripteurs de fichiers

1: (Stdout) Position = 0

Variables globales

const char* quisuisje = "Le pere";

```

int main()
{
  pid_t pid;
  quisuisje = "Le pere";
  pid = fork(); //pid=0
  if(pid == 0){
→ quisuisje = "Le fils";
    printf("Je suis %s", quisuisje);
  }
  else{
    printf("Je suis %s", quisuisje);
    wait(NULL);
  }
  return 0;
}

```


SE Linux/Processus/création (fork)

Exemple 3 à tester

fils.c

```
#include <unistd.h>
#include <sys/types.h>

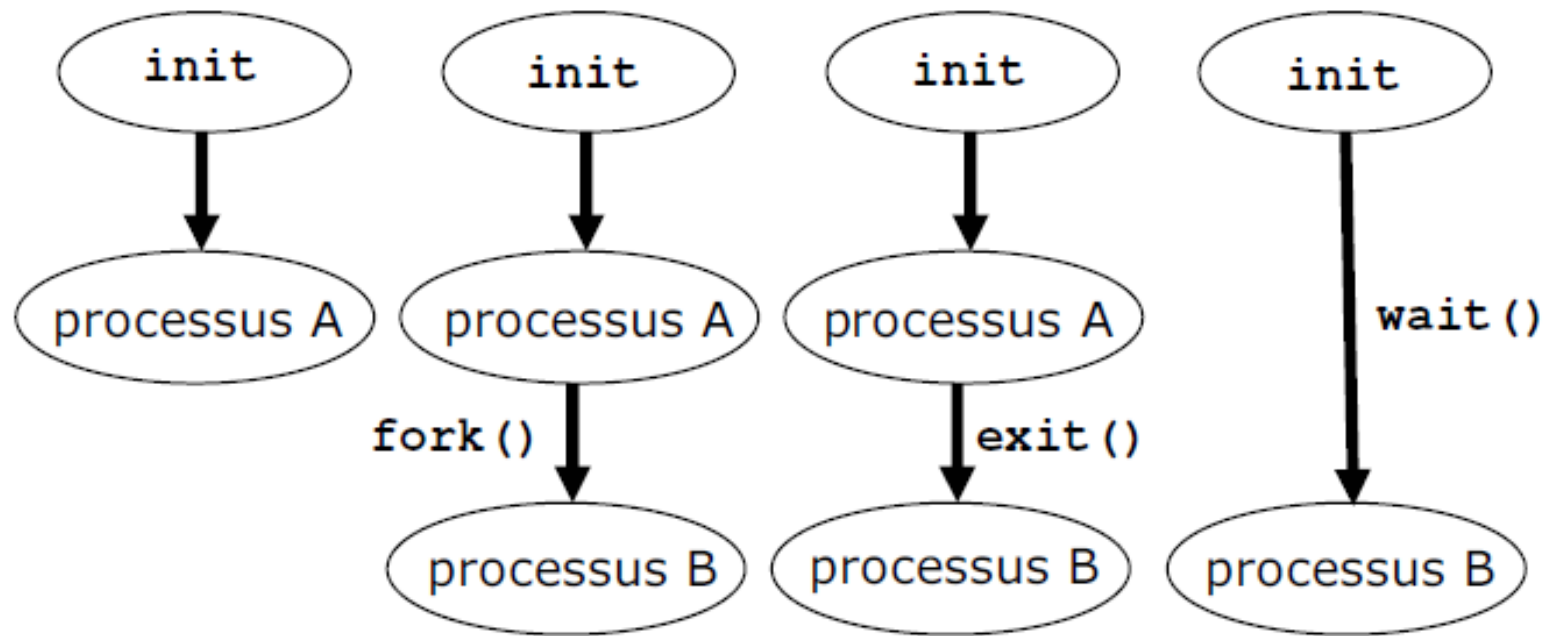
int main()
{
    pid_t fils_pid;
    fils_pid=fork();

    if (fils_pid==0)
        printf("Je suis le fils avec pid %d\n",getpid());
    else if(fils_pid > 0)
        printf("Je suis le pere avec pid %d\n", getpid());
    else
        printf("Erreur dans la creation du fils\n");
}
```

```
zared@zared-VirtualBox:~$ ./fils
je suis le père avec pid 1742
zared@zared-VirtualBox:~$ je suis le fils 1743
```

Résultat d'exécution ./fils

SE Linux/Processus/processus defunct



Processus orphelin : le père meurt avant le fils. `init` adopte l'enfant.

SE Linux/Processus Orphelin

Exemple

Pb: processus père se termine avant son fils qui est encore en exécution

```
zared@zared-VirtualBox:~$ je suis le fils 4123 avec le père 930
je suis le fils 4124 avec le père 930
je suis le fils 4125 avec le père 930
je suis le fils 4126 avec le père 930
```

```
zared@zared-VirtualBox:~$ ./chaîne
je suis le fils 4008 avec le père 4007
je suis le fils 4009 avec le père 4008
je suis le fils 4010 avec le père 4009
je suis le fils 4011 avec le père 4010
zared@zared-VirtualBox:~$ vi chaîne.c
zared@zared-VirtualBox:~$ gcc chaîne.c -o chaîne
zared@zared-VirtualBox:~$ ./chaîne
je suis le fils 4029 avec le père 4028
je suis le fils 4030 avec le père 930
zared@zared-VirtualBox:~$ je suis le fils 4031 avec le père 4030
je suis le fils 4032 avec le père 4031
```

Chaîne.c

```
#include <sys/types.h>
#include <unistd.h>

int main(void)
{
    int    i, j, k,
           n=5;
    pid_t  fils_pid;

    for (i=1; i<n; i++)
    {
        fils_pid = fork();

        if (fils_pid > 0) // c'est le pere
            break;

        printf("Processus %d avec pere %d\n", getpid(), getppid());
    }

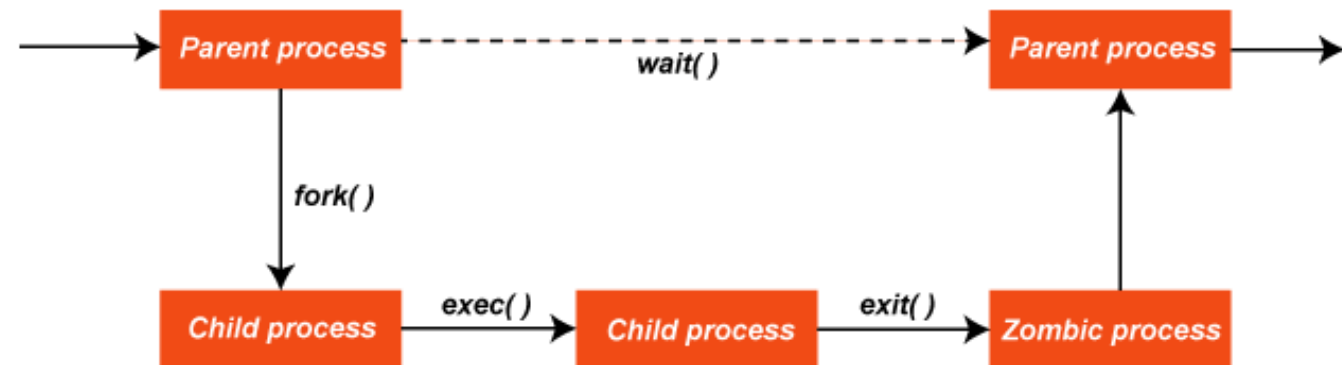
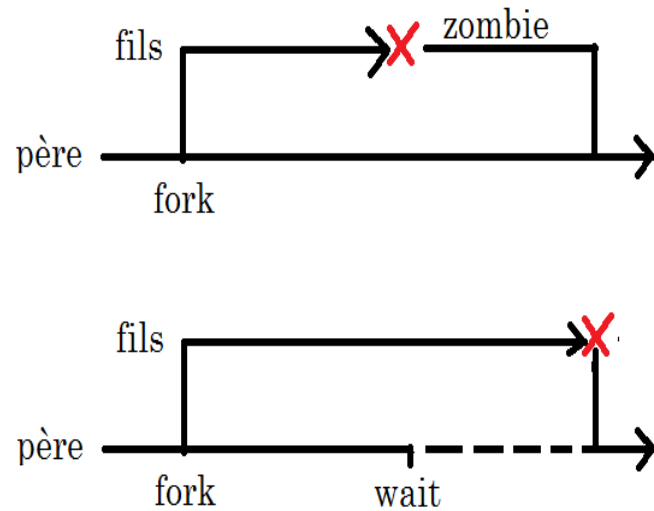
    // Pour faire perdre du temps au processus
    for (j=1; j<100000; j++)
        for(k=1; k<1000; k++);
}
```

SE Linux/Processus/création (fork)

Problème (processus zombie): données de PCB ne seront pas supprimés

```
zared@zared-VirtualBox:~$ ./fils
je suis le père avec pid 1742
zared@zared-VirtualBox:~$ je suis le fils 1743
```

SE Linux/Processus/processus zombie



L'exécution de l'appel système `wait()` ou `waitpid()` par le processus père élimine le fils de la table des processus PCB

SE Linux/Processus/processus defunct

Exemple

Engendrer des zombies est relativement facile.

Le programme zombie.c montre la création d'un processus fils zombie pendant 30 secondes:

zombie.c

```
# include <unistd.h>
# include <stdlib.h>
# include <sys/types . h>
int main ( )
{pid_t pid ;
pid = fork ( ) ; // processus fil s
if ( pid > 0 )
{
sleep (30) ; // Père : dormir 30 secondes
}else
{
exit(0) ; // Fils : quitter immédiatement}
return 0 ;}
```

SE Linux/Processus/ synchronisation de processus

□ **wait(), waitpid() et exit()/<wait.h>**

➤ Processus père attendra la fin d'un de ses processus fils et de récupérer son statut de fin

➤ Appel système wait()

pid = wait(&status); //disponible depuis Linux 2.6.9

int waitpid(int pid, int *status, int options);

SE Linux/Processus/ appels système

□ `waitpid()/<wait.h>`

`int waitpid(int pid, int *status, int options);`

La valeur de pid peut être :

- `< -1` :attendre la fin de n'importe lequel des processus fils dont le GID du processus est égal à la valeur absolue de pid.
- `-1`: attendre n'importe lequel des processus fils.
- `0`: attendre n'importe lequel des processus fils dont le GID du processus est égal à celui du processus appelant.
- `> 0`: attendre n'importe lequel des processus fils dont le PID est égal à pid.

SE Linux/Processus/ appels système

□ `waitpid()/<wait.h>`

```
int waitpid(int pid, int *status, int options);
```

options est un OU binaire entre zéro ou plus des constantes suivantes :

- **WNOHANG**: revenir immédiatement si aucun fils n'est achevé.
- **WUNTRACED**: revenir si un fils est bloqué
- **WCONTINUED** (Depuis Linux 2.6.10): revenir si un fils bloqué a été relancé par la délivrance du signal SIGCONT.

SE Linux/Processus/ appels système

□ waitpid()/<wait.h>

int waitpid(int pid, int *status, int options);

- Si status n'est pas NULL, wait() et waitpid() enregistre les informations sur l'état processus dans l'entier int sur lequel il pointe. Cet entier est analysé avec des macros.

Voir plus:

<http://manpagesfr.free.fr/man/man2/wait.2.html>

SE Linux/Processus/appels système

□ wait(), waitpid() et exit()

Exemple 1 .

le programme fils.c (testé ci-dessus) a le défaut de revenir au shell avant que le fils n'ait pu afficher son message.

```
zared@zared-VirtualBox:~$ ./fils
je suis le père avec pid 1742
zared@zared-VirtualBox:~$ je suis le fils 1743
█
```

En utilisant l'appel wait, on peut facilement corriger ce problème :

fils_wait.c

```
# include <unistd.h>
# include <sys/types.h>
int main ( )
{
    Pid_t fils_pid;
    fils_pid=fork( ) ;
    if (fils_pid ==0)
        printf( " Je suis le fils avec pid %d\n" , getpid ( ) ) ;
    elseif( fils_pid > 0)
    {
        printf ("J e suis le pere avec pid %d.\n" , getpid ( ) ) ;
        printf ( "J ‘attends que mon fils se termine\n" ) ;
        wait (NULL) ;
    }
    else
        printf (" Erreur dans la creation du fils \n");
    exit ( 0 ) ;
}
```

SE Linux/Processus/appels système exec

❏ Remplacer le code dupliqué par fork()

- `int execl( const char *path, const char *arg0, ...,NULL);`
- `int execv(const char *path, char * const argv[]);`
- `int execl( const char *path, const char *arg0, ...,NULL, char * const envp[]);`
- `int execve(const char *file, char * const argv[], char * const envp[]);`
- `int execlp( const char *file,const char *arg0, ... , NULL );`
- `int execvp(const char *file, char * const argv[]);`

SE Linux/Processus/appels système execv

exemple

```
#include <sys/wait.h>
# include <unistd.h>
# include <stdio.h>
# include <sys/types.h>
int main ( int argc , char *argv [ ] )
{Pid_t fils_pid ;
// Liste d'arguments pour la comande " ls "
args[ ] = { "ls", "-l", argv[1], NULL} ;
fils_pid= fork ( ) ;
if(fils_pid!=0 ) {
waitpid (fils_pid ,NULL,NULL) ;
```

```
printf( "Programme principal termine\n" ) ;
return 0 ;
}
else
{
execvp ( " l s " , args ) ;
// Retourner en cas d'erreur
perror ( "Erreur dans execvp" ) ;
exit ( 1 ) ;
}
return 0 ;
}
```

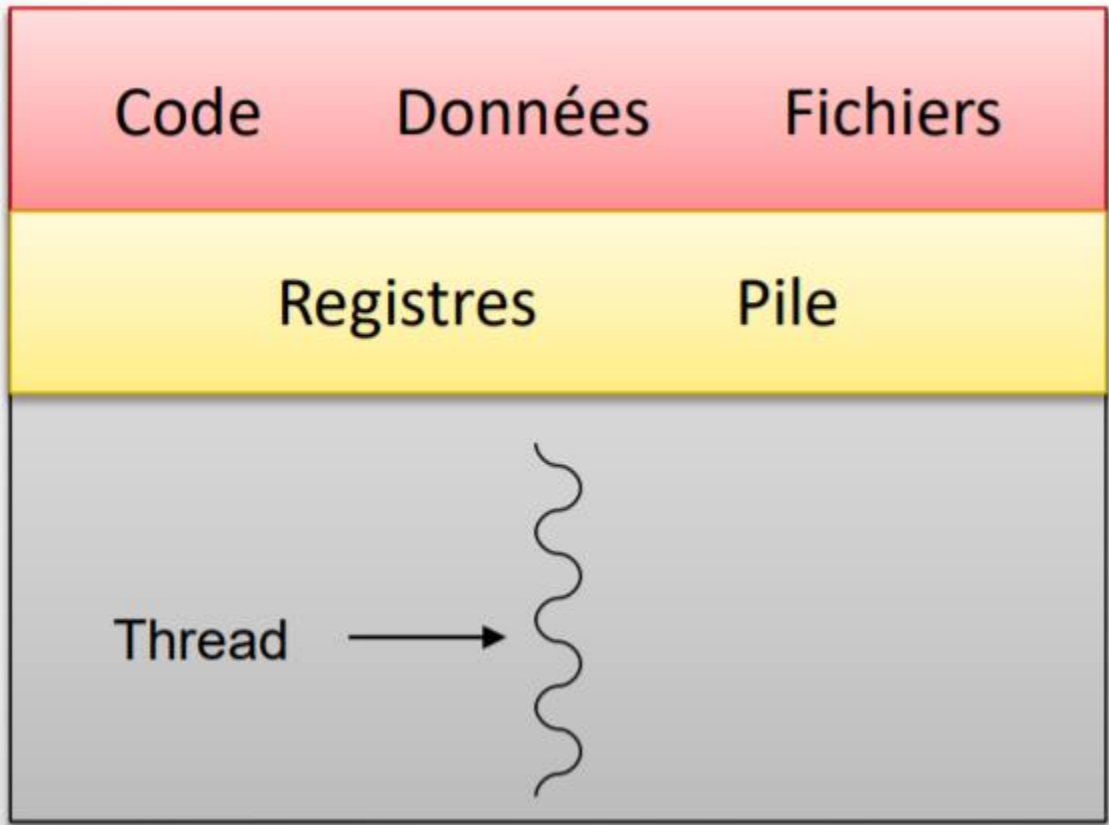
SE Linux/Processus/appels système execv

Les threads pour la gestion des traitements temps réel??

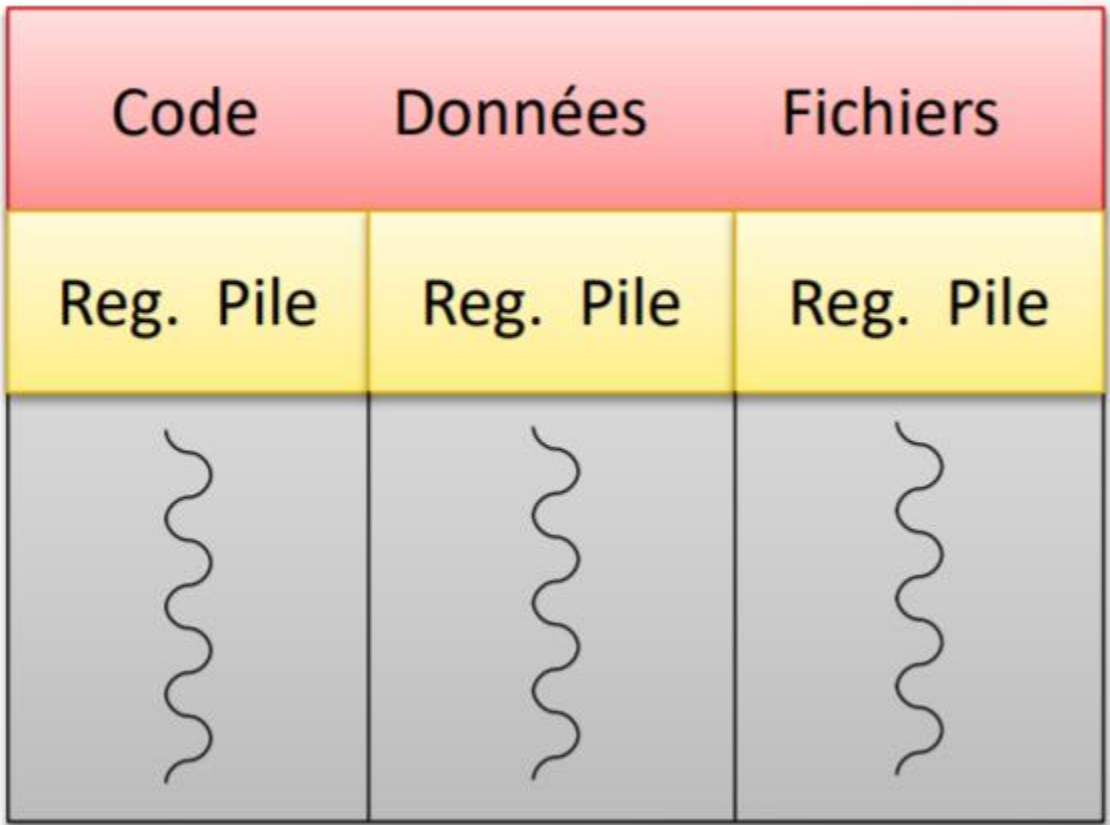


SE Linux/Threads/Processus vs Threads

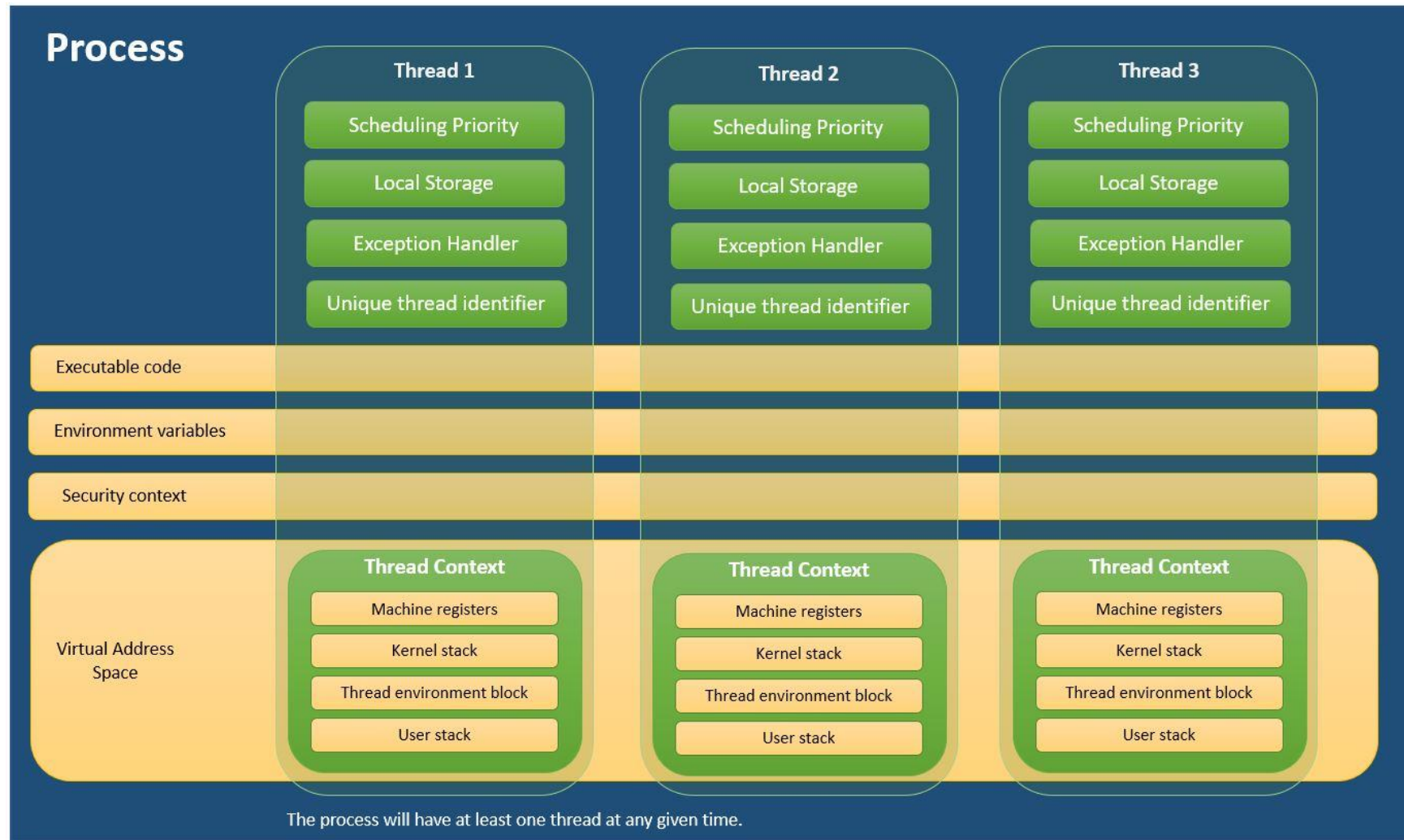
processus



Processus léger



SE Linux/Threads/ Processus vs Thread



SE Linux/Threads/Threads utilisateur et noyau

Le multithreading sont offerts soit au niveau **utilisateur**, soit au niveau **noyau**.

❑ Les threads utilisateur

- Sont supportés au-dessus du noyau et sont implantés par une bibliothèque de threads au niveau utilisateur (Pthread sous linux).
- Ils sont portables sur différentes plate-formes.
- Le changement de contexte est rapide.

❑ Les threads noyau

- Sont directement supportés par le noyau du système d'exploitation.
- Le SE se charge de leur gestion et le changement de contexte est lent.

Bibliothèque Pthread

- ❑ Bibliothèque normalisée POSIX (ISO/IEC 9945-1:1996)
- ❑ Existe sous Linux et ses dérivés, et sous Windows (threads-win32/)
- ❑ Fichier d'en-tête :
`pthread.h`
- ❑ Compilation et édition des liens :
`gcc app.c -lpthread`
- ❑ Déclaration d'un thread
`pthread_t thread;`

SE Linux/Threads/ API/gestion de threads (Posix)

□ Compilation avec la bibliothèque Pthread

```
#include <pthread.h>
```

➤ `gcc -lpthread -o monprogramme monprogramme.c //compilation`

➤ `./monprogramme //exécution.`

SE Linux/Threads/API/ gestion de threads (Posix)

❑ Création de threads

```
int pthread_create (pthread_t *thread , pthread_attr_t *attr, void *nomfonction, void *arg );
```

❑ Suspension de threads

```
int pthread_join(pthread_t *thid, void **valeur_de_retour);
```

❑ Terminaison de threads

```
void pthread_exit(void *valeur_de_retour);
```

❑ Les attributs des threads

```
int pthread_attr_init(pthread_attr_t *attr);
```

initialise la structure d'attributs de thread *attr* et la remplit avec les valeurs par défaut pour tous les attributs

SE Linux/Threads/ API/gestion de threads (Posix)

```
int pthread_create(  
    pthread_t *thread,  
  
    const pthread_attr_t *attr, void  
    *(*start_routine)(void *), void *arg);
```

thread: est un pointeur sur une variable de type pthread_t
argument de sortie!

Attr: permet de définir les attributs du thread (NULL = attributs par défaut)

start_routine: correspond à la fonction qui est exécutée par le thread.

```
void *fonction(void *data);
```

arg est un pointeur passé à la fonction

SE Linux/Threads/API/ gestion de threads (Posix)

Exemple 1:

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
void *func(void *arg) {
    printf("Hello from our first thread!\n");
    return NULL;}
int main(int argc, char *argv[]) {
    pthread_t thread;
    if (pthread_create(&thread, NULL, func, NULL) != 0) {
        perror("thread creation error");
        return EXIT_FAILURE;
    }
    return EXIT_SUCCESS;}
```

SE Linux/Threads/API/ gestion de threads

Exemple 2: Création de thread : passage d'argument

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
void *func(void *arg) {
    char *msg = (char *) arg; printf("Message = %s\n", msg); return NULL;}
int main(int argc, char *argv[]) { pthread_t t;

    char *msg = "Threads are awesome!";
    if (pthread_create(&t, NULL, func, msg) != 0) { perror("thread creation error");

    return EXIT_FAILURE;}

    return EXIT_SUCCESS;}
```

SE Linux/Threads/ API/gestion de threads (Posix)

Exemple: jointure

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
void* fonc(void* arg){
    int i;
    for(i=0;i<7;i++){
        printf("Tache %d : %d\n", (int) arg, i);
        usleep(1000000); //attendre 1 seconde } }
```

thread-pid.c

```
int main(void) {
    pthread_t tache1, tache2; //déclaration des deux
    tâches
    pthread_create(&tache1, NULL, fonc, (void*) 1);
    pthread_create(&tache2, NULL, fonc, (void*) 2);
    pthread_join(tache1, NULL); //main(void), doit
    attendre la fin de l'exécution de la tâche tache1
    pthread_join(tache2, NULL);
    return 0; }
```

Terminaison: options

- ❑ La terminaison d'un thread peut être exécutée depuis:
 - Le thread lui-même:
`return`
`pthread_exit()`
 - Un autre thread:
`pthread_cancel()`
- ❑ Mal terminer un thread peut laisser le système dans un état incohérent!!
 - Plus spécifiquement depuis un autre thread
- ❑ Pour terminer l'application (destruction de tous les threads):
`exit()`
- ❑ `return value;`
 - Attention avec le thread principal: Terminaison du programme!!
- ❑ `void pthread_exit(void *value)`
 - La fonction met fin au thread, et retourne `value` au thread attendant grâce à une jointure.

Terminaison

```
void *tache1(void *arg) {  
    printf("Tâche 1\n");  
    return 3;}  
void *tache2(void *arg) {  
    printf("Tâche 2\n");  
    pthread_exit(4);}  
int main(int argc, char *argv[]) {  
    pthread_t thread1;  
    pthread_t thread2;  
    pthread_create(&thread1, NULL, tache1, NULL);  
    pthread_create(&thread2, NULL, tache2, NULL);
```

```
    void *statut1;  
    void *statut2;  
    pthread_join(thread1, &statut1);  
    pthread_join(thread2, &statut2);  
    printf("Statut1:%d,  
    Status2:%d\n", (int)(intptr_t)statut1, (int)(intptr_t)  
    )statut2);  
    return EXIT_SUCCESS;  
}
```

SE Linux/Threads/ API/gestion de threads (Posix)

□ Attributs d'une tâche/annexe 1

- `pthread_attr_t attr;` // contenu des propriétés de la tâche
- `pthread_attr_init(&attr);` //initialisation de attr aux valeurs par défaut. Obligatoire avant toute manipulation de attr
- `pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);`
// affectation de la propriété détachable à attr
- `pthread_create(&tache1, &attr, fonc, 1);`
- `pthread_attr_destroy(&attr);` // détruire attr pour libérer la mémoire allouée

SE Linux/Threads/ API/gestion de threads (Posix)

❏ Priorité et ordonnancement de tâches

- structure de données Posix utilisée pour affecter une priorité à la tâche à créer :

struct sched_param param

```
struct sched_param {  
    int sched_priority; };
```

- priorité et type d'ordonnancement

int pthread_attr_setinheritsched(pthread_attr_t *attr, int inheritsched)

- Valeur de variable *inheritsched* :

- **PTHREAD_EXPLICIT_SCHED** :

la tâche à créer sera forcée d'utiliser les paramètres d'ordonnancement (dans *attr*) ;

- **PTHREAD_INHERIT_SCHED** :

la tâche héritera des propriétés d'ordonnancement de son processus créateur.

SE Linux/Threads/ API/gestion de threads (Posix)

❑ Priorité et ordonnancement de tâches

➤ Affectation de la priorité à attr

int pthread_attr_setschedparam(pthread_attr_t *attr, **const struct** sched_param *param)

➤ affecter à *attr* de la tâche à créer le type d'ordonnancement noté *policy* :

int pthread_attr_setschedpolicy(pthread_attr_t *attr, **int** policy)

➤ type d'ordonnancement correspondant sous Posix (policy)

- ✓ *SCHED_FIFO* : ordonnancement préemptif à priorités fixes.
- ✓ *SCHED_RR* : ordonnancement Round-Robin à priorité préemptif (quantum de temps).
- ✓ *SCHED_OTHER* : autre ordonnancement.

SE Linux/Threads/ API/gestion de threads (Posix)

□ Priorité et ordonnancement de tâches

- Affecter les paramètres d'ordonnancement à une tâche **en cours d'exécution**.

```
int pthread_setschedparam(pthread_t thread, int policy, const struct sched_param  
                           *param)
```

Exercice1:

Exemple precedent avec prise en compte les paramètres d'ordonnancement

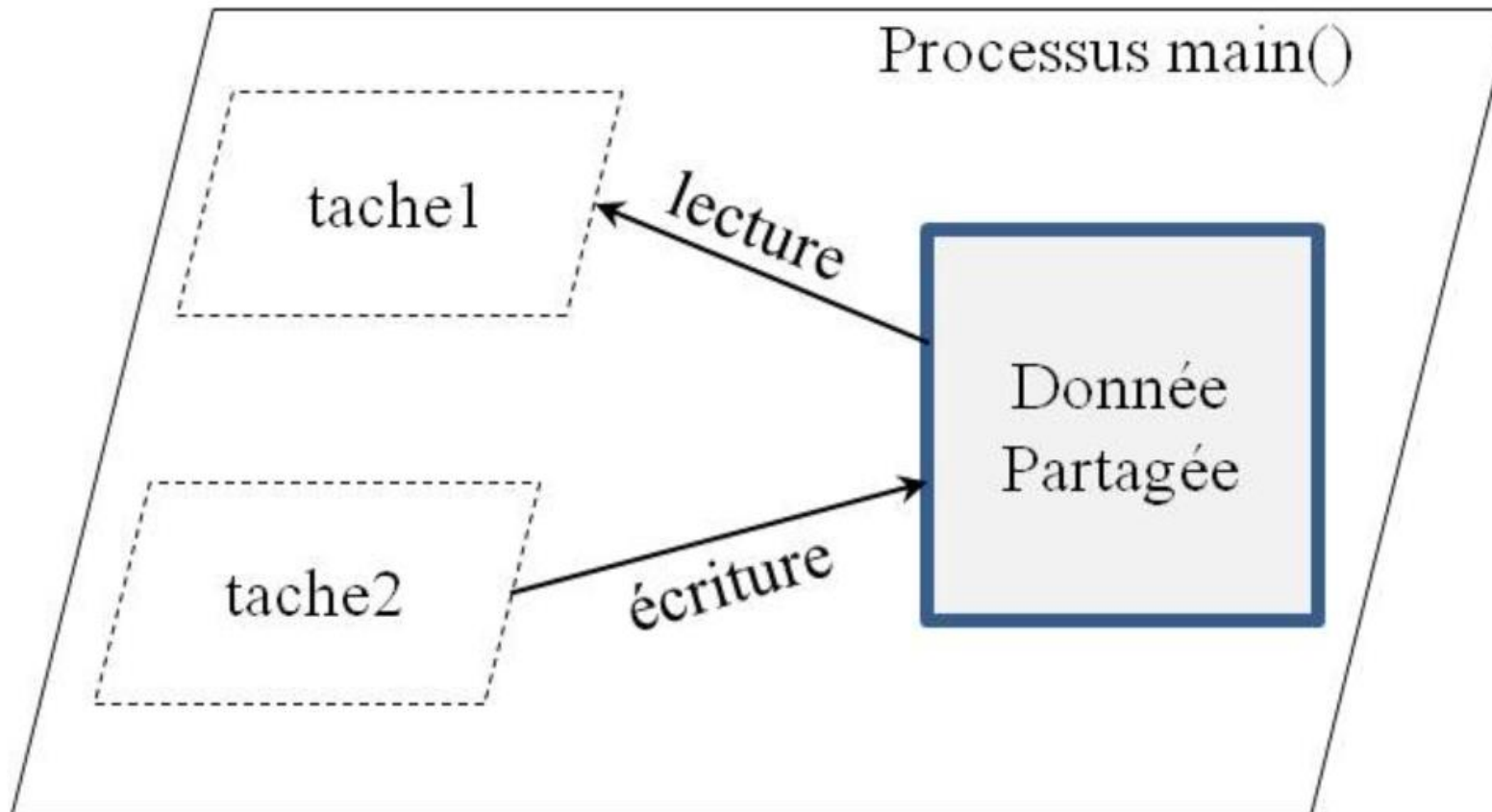
Doc: Execice 1 Word

SE Linux/Threads/ API/gestion de threads (Posix)

Appel	Description
pthread_create	Créer un nouveau fil d'exécution
pthread_exit	Terminer le fil d'exécution appelant
pthread_join	Attendre la fin d'un autre fil d'exécution
pthread_mutex_init	Créer un mutex
pthread_mutex_destroy	Détruire un mutex
pthread_mutex_lock	Verrouiller un mutex
pthread_mutex_unlock	Relâcher un mutex
pthread_cond_init	Créer une variable de condition
pthread_cond_destroy	Détruire une variable de condition
pthread_cond_wait	Attendre après une variable de condition
pthread_cond_signal	Signaler une variable de condition

SE Linux/Threads/ API/partage de données (Posix)

□ Gérer le partage de données entre tâches



SE Linux/Threads/API/ partage de données (Posix)

❑ Exclusion mutuelle

1. Déclaration de donnée/ressource partagée (ex:tout type dans le langage C)
2. Déclaration du *mutex*. C'est le verrou qui va gérer l'accès à la donnée partagée

```
pthread_mutex_t verrou
```

3. Initialiser le verrou. Obligatoire avant toute utilisation de ce verrou.

```
pthread_mutex_init(&pthread_mutex_t *verrou, const pthread_mutexattr_t  
*m_attr)
```

4. La fonction permettant à une tâche de prendre le verrou :

```
pthread_mutex_lock(&pthread_mutex_t *verrou)
```

5. La fonction permettant à une tâche de libérer le verrou après avoir utilisé la donnée partagée :

```
pthread_mutex_unlock(&pthread_mutex_t *verrou)
```


SE Linux/Threads/API/ Inversion de priorité(1/5)

❑ Problème d'inversion de priorité

Cas classique

reposant sur trois tâches T1, T2 et T3 de priorités strictement croissantes ainsi qu'un mutex M.

1. Initialement, T2 et T3 sont endormies, en attente d'événements externes. T1 s'exécute et va accéder à une ressource partagée avec T3. Aussi T1 verrouille-t-elle le mutex M.
2. L'événement externe qu'attendait T3 se produit. La tâche T3 se réveille, et préempte T1 car elle est plus prioritaire. T3 est donc en exécution, T2 endormie et T1 préemptée.
3. La tâche T3 va devoir accéder à son tour à la ressource partagée avec T1. Pour cela elle doit d'abord verrouiller le mutex M, toutefois ce dernier est déjà tenu par T1. T3 va donc s'endormir en attendant la libération de M, et T1 va reprendre son exécution. Jusqu'ici tout se déroule parfaitement normalement.

SE Linux/Threads/ Inversion de priorité(2/5)

□ API thread

4. Mettre dans la propriété *m_attr* qui sera affectée au mutex, sa priorité plafond « *prioceiling* »

```
int pthread_mutexattr_setprioceiling(pthread_mutexattr_t *m_attr, int  
prioceiling)
```

5. mettre dans la propriété *m_attr* qui sera affectée au mutex, son protocole d'héritage *protocol*

```
int pthread_mutexattr_setprotocol(pthread_mutexattr_t *m_attr, int  
protocol)
```

6. La fonction permettant à une tâche disposant de la propriété *attr* d'hériter d'une priorité

```
int pthread_attr_setscope(pthread_attr_t *attr, int scope)
```

Avec scope doit être à:

PTHREAD_SCOPE_SYSTEM, si la tâche est en concurrence avec toute autre tâche de tout processus du système

PTHREAD_SCOPE_PROCESS, si la tâche est uniquement en compétition avec les tâches internes au même processus

SE Linux/Threads/ API/Inversion de priorité(3/5)

❑ Cas classique

T1, T2 et T3 de priorités strictement croissantes.

4. L'événement attendu par T2 survient. Cette tâche va donc se réveiller. Comme elle est plus prioritaire que T1, cette dernière est préemptée et la situation est la suivante:
- T1 préemptée tient le mutex M
 - T2 s'exécute
 - T3 endormie attend le mutex M

La tâche T3 est bloquée par l'exécution d'une tâche T2 moins prioritaire, nous sommes dans une situation d'*inversion de priorité*.

SE Linux/Threads/ Inversion de priorité(4/5)

❑ Solution

le système **PIP** (*priority inheritance protocol*):

est un mécanisme reposant sur deux règles et sur une valeur de priorité appliquée à chaque mutex (et éventuellement à toutes les ressources de synchronisation). Reprenons la situation décrite plus haut, T1 est actif, T2 et T3 dorment. T1 verrouille le mutex.

➤ **Première règle :**

un mutex hérite de la priorité la plus élevée parmi celles de tous les threads qui essayent de le verrouiller

Le thread T1 s'exécutant avec la priorité P1, le mutex hérite alors de cette valeur.

Ensuite T3 se réveille et demande à son tour le mutex. La priorité de celui-ci monte alors à la valeur P3. La seconde règle s'applique

SE Linux/Threads/ Inversion de priorité(5/5)

❑ Solution

le système **PIP** (*priority inheritance protocol*):

➤ **Seconde règle :**

un thread s'exécute avec la priorité la plus élevée parmi celles de tous les mutex qu'il a verrouillé.

Le thread T1 est alors propulsé à la priorité P3. Le thread T2 se réveille, mais sa priorité étant à présent inférieure à celle de T1 il ne peut le préempter, et T1 va s'exécuter intégralement, relâcher le mutex, redescendre alors à la priorité P1. T3 pourra alors prendre le mutex et s'exécuter avant enfin de laisser passer T2.



SE Linux/Threads

pour plus de détails voir

https://xenomai.org/documentation/xenomai-2.6/html/api/group__posix__mutex.html

Annexe 1 : Pthread_attr_t

Attribute	Value	Result
<i>scope</i>	PTHREAD_SCOPE_PROCESS	New thread is unbound - not permanently attached to LWP.
<i>detachstate</i>	PTHREAD_CREATE_JOINABLE	Exit status and thread are preserved after the thread terminates.
<i>stackaddr</i>	NULL	New thread has system-allocated stack address.
<i>stacksize</i>	1 megabyte	New thread has system-defined stack size.
<i>priority</i>		New thread inherits parent thread priority.
<i>inheritsched</i>	PTHREAD_INHERIT_SCHED	New thread inherits parent thread scheduling priority.
<i>schedpolicy</i>	SCHED_OTHER	New thread uses Solaris-defined fixed priority scheduling; threads run until preempted by a higher-priority thread or until they block or yield.

Annexe 1 : Pthread_attr_t/pthread.h

```
typedef struct {  
341  int flags;  
342  #ifdef _POSIX_THREAD_ATTR_STACKSIZE  
343  int stacksize;  
344  #endif  
345  int contentionscope;  
346  int inheritsched;  
347  int detachstate;  
348  int sched;  
349  struct sched_param param;  
350  struct timespec starttime, deadline, period;  
351 } pthread_attr_t;
```


Annexe 1 : Pthread_mutexattr_t

```
#define PTHREAD_MUTEX_NORMAL 0
#define PTHREAD_MUTEX_RECURSIVE 1
#define PTHREAD_MUTEX_ERRORCHECK 2
#define PTHREAD_MUTEX_DEFAULT PTHREAD_MUTEX_NORMAL

#define PTHREAD_PRIO_NONE 0
#define PTHREAD_PRIO_INHERIT 1

#define PTHREAD_PRIO_PROTECT 2
#define PTHREAD_MUTEX_STALLED 0
#define PTHREAD_MUTEX_ROBUST 1

typedef struct
{
    int pshared;
    int kind;
    int protocol;
    int robustness;
} pthread_mutexattr_t;
```

Annexe 2 :

pthread_cond_t

```
typedef struct {
```

```
280   struct pthread_queue queue; //280: numéro de la ligne  
                                     dans le fichier du code pthread.h
```

```
281   int flags;
```

```
282   int waiters;
```

```
283   pthread_mutex_t *mutex;
```

```
284 } pthread_cond_t;
```

Annexe 1 : Pthread_mutexattr_t

PTHREAD_COND_INITIALIZER

```
#define PTHREAD_COND_INITIALIZER { \  
291  { 0, 0 }, /* queue */ \  
292  1, /* flags */ \  
293  0, /* waiters */ \  
294  0 /* mutex */ \  
295 }
```

Annexe 1 : Pthread_mutexattr_t

PTHREAD_mutex_t

```
typedef struct {  
179  struct pthread_queue queue;  
180  char lock;  
181  struct pthread *owner;  
182  int flags;  
183  #ifdef _POSIX_THREADS_PRIO_PROTECT  
184  int prioceiling;  
185  pthread_protocol_t protocol;  
186  int prev_max_ceiling_prio;  
187  #endif  
188  } pthread_mutex_t;
```



RTEMS Autre STR

- ❑ RTEMS = Real Time Executive for Multiprocessor Systems
 - Exécutif temps-réel embarqué diffusé sous licence libre (GPL avec exception)
 - Pas exactement un système d'exploitation car mono-application (mais multi-thread)/ Military Systems
 - Programmation C, C++, Ada
 - Plus de 100 BSP disponibles pour 20 architectures
 - API RTEMS « classique » ou POSIX